Decision 모델 종합 리포트

초보자를 위한 원리 설명과 모델 구조 비교
로컬 문서 분류 앱의 사용법과 코드 해설

2026년 10월 4일 · 영상과 GeekNews 추가 글을 통합한 분석

먼저 알아둘 핵심

Decision 모델은 문서를 읽고 정해진 선택지마다 점수를 매기는 AI입니다. 고객 문의를 담당 부서에 보내거나, 외부에 공유하려는 문서에 기밀 정보가 있는지 확인하는 데 활용할 수 있습니다. 긴 답변 대신 프로그램이 바로 다룰 수 있는 결과를 돌려주는 것이 핵심입니다.

다만 Decision은 하나의 고정된 신경망 구조 이름이 아닙니다. 일반 언어 모델의 답 토큰 점수를 읽는 방법, 분류 전용으로 학습한 모델, 별도의 판단 헤드를 붙인 모델, 인코더로 선택지를 평가하는 모델이 모두 이 범주에 들어갑니다.

영상은 작은 로컬 모델의 활용 가능성을 보여주고, 추가 글은 큰 일반 LLM도 출력 방식을 바꾸면 판단에 쓸 수 있음을 보여줍니다. 두 자료를 함께 보면 모델의 이름보다 “어떤 입력을 읽고, 어떻게 점수를 꺼내고, 어떤 데이터로 검증했는가”가 중요해집니다.

이 리포트의 읽는 순서

알고 싶은 것	읽을 부분
처음 듣는 개념을 이해하고 싶다	1–6장 · 판단 방식과 두 자료의 해석
모델들이 내부에서 어떻게 다른가	7–15장 · 17개 계열과 주요 변형
어떤 모델이 내 업무에 맞는가	16–18장 · 보정, 평가 지표, 실제 측정
앱을 쓰고 코드를 수정하고 싶다	19–22장 · 화면 사용법과 구현 구조
용어와 원문을 다시 확인하고 싶다	23–25장 · 용어와 참고 자료

현재 앱은 기밀 문서 검토에 맞춰 Qwen3.5 4B, JevK5 4B, Laya 다국어, decider 0.8B를 실제 로컬 실행합니다. 리포트에서 다루는 나머지 모델은 구조 비교 대상이며 앱에 모두 설치된 것은 아닙니다.

1 판단 모델을 일상 업무에 대입하기

우편 분류 담당자를 떠올려 보세요. 편지를 읽고 회계팀, 기술지원팀, 고객상담팀 중 어디로 보낼지 고릅니다. 좋은 담당자는 이름 하나만 고르는 데 그치지 않고, 애매한 편지는 사람이 더 살펴보도록 남깁니다. 판단 모델과 앱의 역할도 비슷합니다.

입력에는 세 가지가 필요합니다

요소	쉬운 뜻	문서 검토 예시
상태 State	판단할 원문이나 현재 상황	아직 발표하지 않은 계약 할인율이 적힌 메모
질문 Instruction	무엇을 판단하는지 정한 기준	실제 비공개 정보가 포함되어 있는가
선택지 Options	고를 수 있는 답과 그 의미	예는 기밀 포함, 아니요는 기밀 없음

선택지의 뜻은 모델 이름만큼 중요합니다. “대외비라는 단어가 있나?”는 문자 검색에 가까운 질문이고, “실제 비공개 정보가 있나?”는 문맥을 해석해야 하는 질문입니다. 공개 보안 교육 자료에는 대외비라는 단어가 많아도 실제 기밀이 없을 수 있습니다.

모델의 답과 회사의 행동은 다른 단계입니다

예를 들어 AI가 기밀 판단 점수 0.45를 냈다고 합시다. 앱의 검토 시작 기준이 0.25, 차단 기준이 0.60이면 사람 검토가 됩니다. 차단 기준을 0.40으로 내리면 같은 문서와 같은 점수도 반출 금지가 됩니다. 앞 단계는 AI의 판단이고 뒷 단계는 사람이 정한 운영 규칙입니다.

정해진 선택지 밖의 답이 필요할 수도 있습니다. 문의를 회계팀과 기술팀 중 하나로만 보내게 하면 어느 쪽에도 속하지 않는 문의도 억지로 배정됩니다. 범용 분류 앱을 만들 때는 “해당 없음”이나 “사람 확인”을 명시적인 선택지 또는 별도 정책으로 설계해야 합니다.

이 앱이 맡는 범위

이 앱은 문서를 외부로 보내거나 승인하지 않습니다. 가능 후보, 사람 검토, 반출 금지라는 검토 결과를 보여줍니다. 점수가 낮은 가능 후보도 실제 승인과 같지 않습니다. 현재 질문과 지표는 기밀 여부에 맞춰져 있으므로, 다른 업무로 확장하려면 결과 이름과 평가 방식도 함께 바꿔야 합니다.

2 글을 쓰지 않고 점수만 읽는 원리

언어 모델은 문장을 한 번에 완성하지 않습니다. 지금까지의 문맥을 읽고 다음 토큰의 점수를 계산한 뒤, 토큰을 하나 붙이고 다시 다음 것을 계산합니다. 토큰은 글자를 쪼개 놓은 조각입니다. 한국어 한 글자와 정확히 일치하지 않을 수 있습니다.

필요한 것이 두 선택지뿐이라면

문서 뒤에 A는 기밀, B는 비기밀이라고 알려줍니다. 다음 답을 계산하는 순간, 모델 안에는 A와 B뿐 아니라 다른 수많은 토큰의 점수가 있습니다. 그중 A와 B의 값만 꺼내면 긴 설명을 만들지 않고도 두 답 중 어느 쪽을 더 선호하는지 알 수 있습니다.

예시 계산	값 또는 뜻
전체 어휘에서 A 확률	0.30
전체 어휘에서 B 확률	0.10
A와 B 안에서 A의 상대 비중	0.30 ÷ (0.30 + 0.10) = 0.75
올바른 해석	제시한 두 답 사이에서는 A 쪽 점수가 높다
바로 결론 내릴 수 없는 것	실제 기밀일 확률이 정확히 75퍼센트다

이 예에서는 나머지 어휘가 0.60을 차지합니다. 모델이 A/B 형식을 제대로 따르지 않았을 수 있습니다. 현재 앱의 일반 LLM 방식은 두 선택지의 원래 확률 합도 확인하고, 합이 0.5보다 작으면 사람 검토로 남깁니다. 모든 구현이 이 검사를 하는 것은 아닙니다.

한 번 계산한다는 말의 범위

입력 전체를 읽는 계산인 prefill은 여전히 필요합니다. 없애는 것은 긴 답을 한 조각씩 쓰는 반복 생성입니다. 따라서 긴 문서를 넣으면 느려질 수 있고, 수십 개 질문을 따로 평가하면 추가 계산이 생깁니다. 한 번의 API 호출, 한 번의 모델 계산, 여러 질문의 동시 처리는 서로 같은 말이 아닙니다.

첫 실행에는 모델 파일을 메모리에 올리는 준비 시간도 걸립니다. 이미 올라온 모델의 짧은 문서 처리 시간과, 앱을 처음 켜고 결과를 받기까지의 시간은 구분해서 봐야 합니다.

관련 구현: [S3] Privatemode Decisions. 이 장의 A/B 수치는 원리를 설명하기 위한 자체 예시입니다.

3 추가 글에서 새롭게 배울 점

GeekNews가 소개한 원문은 GLM-5.3-Flash를 별도 재학습 없이 판단 모델처럼 사용하는 실험입니다. 질문과 선택지를 넣고 답의 시작 부분을 미리 채운 뒤, 다음 위치에서 선택지 번호의 확률을 읽습니다. 모델을 새로 만든 것이 아니라 기존 모델을 호출하고 해석하는 방식을 바꿨습니다. [S2]

구현에서 놓치기 쉬운 부분

답의 시작을 미리 채우는 것을 assistant prefill이라고 합니다. 원문의 설명은 choice_index: 뒤를 이어 쓰게 합니다. 현재 라이브러리 문서의 예시는 answer:를 사용하므로, 구현할 때는 특정 문자열을 외우기보다 실제 버전의 프롬프트 계약을 그대로 따라야 합니다. [S2, S3]

번호 12가 반드시 토큰 하나인 것은 아닙니다. 토큰 번호도 모델마다 다릅니다. 라이브러리는 실제 서버의 토크나이저를 통해 후보 번호가 어떤 토큰인지 확인합니다. 또한 출력 가능한 토큰을 제한하는 설정과, 원하는 후보의 확률을 돌려받는 설정을 구분합니다. [S3]

특히 vLLM의 일반 상위 토큰 목록만 받으면 공백 같은 형식 토큰 때문에 필요한 선택지가 빠질 수 있습니다. 원문은 logprob_token_ids로 후보 토큰의 로그확률을 직접 요청합니다. 현재 앱의 Ollama 방식은 상위 목록에서 A/B를 찾으므로, 둘 중 하나가 없을 때 결과를 만들어 내지 않고 오류를 냅니다. [S2]

실험 결과에서 읽을 수 있는 범위

보고된 내용	해석
텍스트 28개 데이터셋	GLM과 Jev가 각각 10개에서 앞서고 8개는 1퍼센트포인트 이내였다
차이 중앙값 0.7퍼센트포인트	Jev 쪽 차이지만, 유의한 차이를 검출하지 못한 결과가 모든 업무에서 동등함을 증명하지는 않는다
영어와 독일어의 공개 평가	한국어 회사 기밀 문서에 대한 직접 검증은 아니다
클라우드 왕복 시간을 포함한 속도	서버 위치와 네트워크가 달라지면 순서도 달라질 수 있다

이 결과는 큰 일반 모델에서도 빠른 판단 인터페이스를 만들 수 있다는 근거입니다. 한편 원문의 Laya는 약 4.21억 매개변수 영어판이며, 현재 앱의 약 3.22억 다국어판과 구분해야 합니다. [S2, S4, S10]

원문은 2026년 9월 24일 게시물입니다. 가격과 지연 시간은 그 실험 조건의 보고값이며 현재 견적이나 이 PC의 측정값으로 사용하지 않습니다.

4 영상과 추가 글을 함께 해석하기

영상의 질문은 “작은 로컬 모델로 한국어 기밀 문서를 걸러낼 수 있나?”에 가깝습니다. 추가 글의 질문은 “큰 일반 언어 모델을 재학습 없이 빠른 선택지 판단에 쓸 수 있나?”입니다. 목표와 데이터, 실행 위치가 다르므로 한쪽의 성적표를 다른 쪽에 붙여서는 안 됩니다.

구분	영상	추가 글
중심 상황	내 PC에서 한국어 문서 판단	호스팅된 일반 LLM의 판단 활용
평가 내용	기밀 여부라는 한 업무	문의 분류 등 여러 공개 과제
데이터	가상 문서 200건	텍스트 28개와 이미지 1개 데이터셋
중요한 한계	기준 조정·채점 문서가 같음	언어·선택지·서버 환경이 실제 업무와 다름
실무에 가져올 점	작은 모델도 먼저 시험해 볼 수 있음	긴 설명 생성이 판단의 필수 조건은 아님

영상의 95퍼센트를 정확히 읽기

영상에서 Qwen 4B와 JevK5는 기밀 60건 중 57건을 찾았습니다. 이것이 탐지율 95퍼센트입니다. 공개 140건 중 8건을 기밀로 잘못 보아 오탐률은 약 5.7퍼센트입니다. 전체 문서의 정확도 95퍼센트나, 통과한 문서가 95퍼센트 안전하다는 뜻이 아닙니다. [S1]

또한 쉬운 공개 문서 100건에서는 오탐이 없지만, 기밀 표시가 등장하는 공개 문서 26건에서는 8건을 잘못 걸렀습니다. 내 회사에 교육 자료나 인용문이 많다면 전체 평균보다 이 어려운 부분의 결과가 더 중요할 수 있습니다. [S1]

두 자료에서 공통으로 얻는 실무 원칙

첫째, 답의 형식을 고정하면 앱에 연결하기 쉬워집니다. 둘째, 빠른 답이 항상 정확한 답은 아닙니다. 셋째, 점수의 보정과 운영 기준은 별도로 검증해야 합니다. 넷째, 설치한 모델과 실행기의 버전, 질문과 선택지, 문서 길이를 함께 기록해야 같은 비교를 다시 할 수 있습니다.

영상에서 사용한 Qwen 기준 0.32와 JevK5 기준 0.45는 해당 실험의 설정입니다. 이 앱의 시작값 0.25와 0.60도 보편적인 정답은 아닙니다. 기준값을 정하는 문서와 최종 성능을 확인하는 문서를 나누는 것이 다음 단계입니다.

5 모델 구조를 읽는 네 가지 기준

모델을 볼 때는 본체, 읽어내는 장치, 학습 방법, 실행 방법을 따로 확인하세요. 본체가 같아도 마지막 장치가 다를 수 있고, 가중치가 같아도 질문 형식과 점수 보정이 달라 결과가 바뀔 수 있습니다.

기준	쉬운 비유	실제로 확인할 것
본체 Backbone	문서를 읽는 직원	Qwen, Gemma, ModernBERT 등 기반 모델
헤드 Head	읽은 내용을 점수로 변환	어휘 점수, 선택지 포인터, 분류 헤드
학습 Training	직무 교육 방식	정답 학습, LoRA, 전체 학습, 강화 학습
실행 Runtime	업무 처리 환경	템플릿, 토큰화, 압축, 보정, 입력 한도

인코더와 디코더

인코더는 입력 문장의 앞뒤 문맥을 함께 읽어 표현을 만듭니다. 분류에 잘 맞으며 Laya와 Von이 이 계열입니다. 디코더는 앞선 내용으로 다음 토큰을 계산하는 구조이고 Qwen 기반 모델들이 이 계열입니다. 디코더도 꼭 긴 글을 만들 필요는 없습니다. 마지막 상태나 답 토큰 점수만 읽으면 판단기로 사용할 수 있습니다.

Dense와 MoE

Dense는 대부분의 주요 가중치를 매 토큰 계산에 사용하는 형태입니다. MoE는 여러 전문가 모듈 중 일부를 골라 쓰는 방식입니다. 큰 도서관에서 관련 담당자만 일하는 것과 비슷하지만, 전체 전문가의 가중치를 보관하는 비용까지 사라지는 것은 아닙니다. 총 매개변수와 활성 매개변수를 구분해야 합니다.

Attention과 혼합 구조

Attention은 지금 읽는 부분이 다른 부분을 얼마나 참고할지 계산합니다. Full attention은 넓게 서로를 비교하고, 선형 방식은 누적 상태 등을 이용해 긴 입력의 계산 부담을 줄입니다. Qwen3.5 소형 계열은 GatedDeltaNet과 full attention을 섞습니다. 이것은 MoE와 다른 축의 설계입니다. 혼합 attention이라고 곧 MoE인 것은 아닙니다. [S7]

아래 모델 설명에서는 이런 축을 분리합니다. 이름에 4B가 붙었다고 파일이 4GB인 것도 아닙니다. 시각 모듈 포함 여부, 숫자의 정밀도, 실행 중 메모리에 따라 실제 저장 용량과 필요 메모리는 달라집니다.

6 판단을 가르치고 점수를 다듬는 방법

정답 학습과 LoRA

지도 학습은 문서와 올바른 답을 보여주며 틀린 쪽 점수를 낮추고 맞는 쪽을 높이는 교육입니다. 전체 미세 조정은 본체의 많은 가중치를 바꿉니다. LoRA는 작은 추가 행렬을 학습하여 비교적 적은 학습 자원으로 동작을 바꾸는 방법입니다. 학습 후 본체와 합친 merged 가중치를 배포할 수도 있습니다.

따라서 LoRA 모델이라는 말은 “작은 모델”이라는 뜻이 아닙니다. 9B 본체에 작은 LoRA 파일을 붙이면 실행에는 여전히 9B 본체가 필요합니다. LoRA만 받은 파일과 본체가 합쳐진 전체 가중치를 구분해야 합니다.

포인터 헤드와 답 토큰 읽기는 다릅니다

답 토큰 방식은 기존 어휘에서 A, B 같은 답의 점수를 꺼냅니다. 포인터 헤드는 선택지 설명이 끝나는 위치의 내부 표현과 질문의 표현을 비교합니다. 답을 가리키는 전용 장치를 붙이는 셈입니다. Kev와 Eos는 전용 헤드를 사용하지만 계산식과 구성은 같지 않습니다. [S12, S13]

RLCD는 학습 목표에 관한 말입니다

RLCD는 정답만 맞히는 것뿐 아니라, 얼마나 확신하는지도 잘 표현하도록 보상하는 판단 학습 방향입니다. TypeSafe는 이 목표를 Jev의 특징으로 설명합니다. 이는 Transformer 층 수나 인코더 여부를 직접 알려주는 이름이 아닙니다. RLCD를 썼다는 사실만으로 두 모델의 본체 구조가 같다고 볼 수 없습니다. [S5, S6]

예를 들어 맞을 가능성이 애매한 문제에 항상 0.99라고 답하면, 한 번 크게 틀렸을 때 나쁜 점수를 받게 하는 학습을 생각할 수 있습니다. 그러나 학습 데이터에서 잘 맞춘 눈금도 다른 회사 문서에서는 어긋날 수 있습니다. 보정된 모델이라는 설명이 실무 검증을 대신하지는 못합니다.

보정 온도는 출력 생성 온도와 다릅니다

보정 온도 T는 계산된 선택지 점수의 차이를 줄이거나 늘리는 숫자입니다. 양수의 T로 모든 선택지 점수를 나누고 softmax를 하면 가장 큰 선택지는 그대로지만 확률 분포가 바뀝니다. 반면 생성 온도는 어떤 토큰을 뽑을지에 관련됩니다. “temperature=0이니 확률도 완벽하고 결과도 항상 같다”는 결론은 성립하지 않습니다.

7 일반 LLM을 판단에 쓰는 두 사례

Qwen3.5 4B

구조 Dense 기반의 혼합 attention 본체 → 기존 어휘 헤드 → A/B 점수

4B 텍스트 본체는 32개 층에서 GatedDeltaNet 세 층과 full attention 한 층을 반복하는 형태입니다. 일반 언어·시각 작업을 위한 모델이지만, 현재 앱은 텍스트만 넣고 첫 답 토큰의 점수만 읽습니다. Decision용으로 추가 학습한 가중치는 아닙니다.

사용할 때 기준 모델로 두면 추가 판단 학습의 효과를 비교하기 좋습니다. 다만 앱의 Q4 압축본과 JevK5의 Q8 압축본은 조건이 달라, 차이를 학습의 효과 하나로만 설명할 수는 없습니다. 4B라는 모델명과 배포 파일의 전체 매개변수 메타데이터도 같지 않을 수 있습니다.

근거 [S7]

GLM 5.3 Flash

구조 멀티모달 MoE 본체 → 일반 어휘 점수 → 선택지 번호만 추출

공식 모델 카드는 총 약 320B, 활성 약 18B를 제시합니다. sparse attention과 linear attention의 혼합 및 mHC를 사용합니다. 추가 글의 라이브러리는 이 본체를 다시 학습하지 않고, 번호로 된 선택지의 다음 토큰 확률을 읽습니다. 읽는 방식이 바뀌었을 뿐 소형 모델로 바뀐 것은 아닙니다.

사용할 때 이미지를 포함한 판단도 가능한 기반이지만 작은 노트북용 4B 모델의 대체 파일로 생각하면 안 됩니다. 활성 18B라는 수치도 전체 가중치가 18B만 필요하다는 뜻이 아닙니다. 글의 호스팅 환경과 로컬 실행 가능성을 구분해서 검토해야 합니다.

근거 [S8, S2, S3]

두 모델에서 공통으로 확인할 것

모델이 지원하는 기능과 현재 앱이 사용하는 기능을 구분하세요. 기반 모델이 이미지를 읽을 수 있어도 앱이 텍스트만 받으면 이미지 분류 앱은 아닙니다. 또한 선택지 토큰의 확률을 제공하는 실행기인지, 실제 프롬프트 위치에서 토큰이 하나인지 확인해야 합니다.

8 Jev와 JevK5는 서로 다른 모델

Jev TypeSafe

구조 공개된 판단 인터페이스와 RLCD 학습 방향 · 구체적인 본체는 비공개

Jev는 상태와 질문, 선택지를 받아 선택지 분포를 반환하는 서비스입니다. TypeSafe는 텍스트 생성보다 판단과 보정된 확률을 목표로 하는 RLCD를 설명합니다. 공개 문서만으로 정확한 기반 모델, 매개변수 수, attention 구성이나 헤드의 구현을 확정할 수는 없습니다.

사용할 때 API의 모양이 같다고 다른 오픈 모델이 Jev의 가중치나 구조를 복제한 것은 아닙니다. 현재 앱에 Jev 서비스 자체는 연결하지 않았습니다. 로컬에서 실행 중인 JevK5와 이름을 구분해야 합니다.

근거 [S5, S6]

JevK5

구조 Qwen3.5 계열 → LoRA 판단 학습을 합친 가중치 → 답 토큰 점수와 보정

독립 개발 모델입니다. 4B v0.3은 Qwen3.5-4B를 바탕으로 작은 LoRA 업데이트를 학습하고 합친 형태입니다. 별도 포인터 헤드 대신 답을 나타내는 토큰의 점수를 읽습니다. 4B 모델 카드의 보정 온도는 1.22입니다. 이 숫자를 모든 크기나 버전에 일괄 적용하면 안 됩니다.

사용할 때 2B와 9B 등 변형이 있으며 선택지 수에 따른 실행 절차도 버전에 따라 다릅니다. 앱은 Ollaya의 전용 실행을 사용하여 템플릿과 보정을 맡깁니다. 가중치만 일반 채팅 모델처럼 불러오는 것과 동일한 구현이 아닙니다.

근거 [S9]

이름보다 본체와 실행 계약을 확인하세요

Jev는 서비스와 학습 방향을, JevK5는 구체적인 공개 기반 모델과 추가 학습을 확인할 수 있습니다. 이름이 비슷하다는 이유로 라이선스, 한국어 성능, 보정값, 입력 한도를 공유한다고 판단하면 안 됩니다. 실험 기록에는 이름뿐 아니라 배포 식별값과 실행 도구도 남겨야 합니다.

9 Laya와 decider의 구조 차이

Laya 영어판과 다국어판

구조 인코더 → 선택지별 MASK 위치 → 분류 헤드 → 질문별 softmax

영어판은 ModernBERT-large에 판단 헤드를 붙인 약 421M 모델입니다. 헤드에는 두 Transformer 층과 선택지 점수 장치, 행동·상위 검토 장치가 설명되어 있습니다. 다국어판은 mmBERT-base 기반 약 322M입니다. 선택지마다 표시 위치의 표현을 읽어 점수를 만들며 RLCD와 온도 보정을 사용합니다.

사용할 때 추가 글의 영어판과 앱의 다국어판을 혼동하지 마세요. 현재 앱의 실행 프로필은 짧은 입력용입니다. 원 개발 구현은 다국어판의 입력 길이 확장을 지원하지만, 실행기 설정과 문서가 실제로 잘리지 않았는지 확인해야 합니다. “글을 생성하지 않는다”는 말은 분류 오류가 없다는 뜻이 아닙니다.

근거 [S10]

decider 계열

구조 Qwen3.5 본체의 판단용 미세 조정 → 답 위치의 후보 토큰 점수

앱의 0.8B는 작은 Qwen3.5 계열을 판단 데이터로 전체 미세 조정한 모델입니다. 긴 설명 대신 선택지, 참·거짓, 순서 있는 점수를 반환하도록 설계했습니다. 계열에는 2B, 4B와 35B-A3B 등도 있으며, 모든 크기가 같은 학습 단계나 입력 기능을 갖는다고 가정하면 안 됩니다.

사용할 때 0.8B 배포 카드의 언어 범위는 영어입니다. 한국어 예제에서 답을 낸다는 사실과 한국어 업무를 검증했다는 사실은 다릅니다. 큰 MoE 변형은 활성 크기만 보고 메모리를 계산하지 말고 전체 가중치와 실행 방식도 확인해야 합니다.

근거 [S11]

두 모델의 속도가 다른 이유

본체 크기뿐 아니라 인코더와 디코더의 계산 방식, 문서 토큰 수, ONNX 또는 GGUF 실행, CPU와 GPU 사용이 함께 영향을 줍니다. 구조가 다르므로 파일이 작은 쪽이 반드시 더 빠르다고 미리 결론 내릴 수 없습니다.

10 선택지를 가리키는 Kev와 Eos

Kev

구조 Qwen3.5 Base + LoRA → 질문과 선택지 표현을 비교하는 포인터 헤드

4B 배포는 Qwen3.5-4B-Base에 rank 16 LoRA와 포인터 헤드를 붙입니다. 각 선택지의 끝 토큰과 질문의 마지막 토큰을 두 투영 장치로 변환해 비교하고, 그 점수를 softmax로 확률화합니다. 기존 어휘에서 A/B 확률을 꺼내는 것과 다른 읽기 장치입니다.

사용할 때 헤드와 템플릿, 보정값을 본체와 함께 로드해야 합니다. 4B 카드의 T=2.41도 해당 체크포인트의 값입니다. 크기가 다른 Kev 변형이나 다른 정밀도에 그대로 복사할 숫자가 아닙니다. 여러 질문 처리가 한 번에 된다는 표현도 해당 런타임 기준으로 확인하세요.

근거 [S12]

Decision 1.0 Eos 0.8B

구조 Qwen3.5 텍스트 본체 → 후보 끝 표현과 질문 표현 → 전용 공유 헤드

공개 구조에는 약 753M 매개변수가 기재되어 있습니다. 24개 층 중 18개는 GatedDeltaNet, 6개는 full attention입니다. 원 모델의 시각 타워와 어휘 생성 헤드는 제외합니다. 후보와 질문의 표현을 256차원으로 비교하는 가지와 작은 신경망 가지의 점수를 합칩니다.

사용할 때 질문마다 유효한 후보만 남겨 보정하고 softmax합니다. 같은 헤드가 참·거짓, 선택, 순서 점수를 처리합니다. 원 구조는 후보 순서가 바뀌어도 결과가 같다고 보장하지 않습니다. 따라서 선택지 순서를 섞는 평가도 필요합니다.

근거 [S13]

헤드를 바꾸면 무엇이 달라질까요

큰 어휘 전체에서 답을 고르는 장치 대신, 이번 질문의 후보들만 비교하는 장치를 사용할 수 있습니다. 이는 형식을 통제하기 쉽게 만드는 설계입니다. 그러나 어느 후보가 사실에 맞는지 알아내는 능력은 본체와 학습 데이터, 질문의 품질에도 달려 있습니다.

11 Jeff와 Winnow의 판단 학습

Jeff Firelex 프로젝트

구조 소형 언어 모델의 판단 학습 → 답 읽기 → 보정 · 업무별 LoRA 추가

Qwen3.5 0.8B·2B와 Gemma 4 E2B 기반 변형이 소개되어 있습니다. 상태와 선택지로 판단하도록 학습하고, 별도의 업무별 LoRA 어댑터를 선택해 문의 분류나 도구 선택 같은 일을 더 학습시킬 수 있습니다. 기본 모델과 추가 어댑터는 별개로 관리됩니다.

사용할 때 이름이 같은 다른 Jeff 프로젝트와 혼동하지 않도록 저장소와 버전을 기록하세요. 기반 버전이 바뀌면 기존 어댑터가 그대로 호환되지 않을 수 있습니다. 개발자가 제시한 특정 업무 성능이 일반 기밀 분류의 성능을 보장하지 않습니다.

근거 [S14]

Winnow E4B와 12B

구조 Gemma 기반 판단 LoRA를 합친 모델 → 요청한 답 토큰 행만 평가

E4B는 Gemma 4 E4B IT를 바탕으로 rank 32 LoRA를 합친 배포입니다. 판단 경로에서는 후보 토큰 점수를 읽고, 일반 채팅용 생성 헤드도 유지합니다. 실행기는 공통 상태를 읽은 계산을 여러 질문에서 재사용할 수 있습니다. 이미지 입력에는 맞는 시각 projector가 필요합니다.

사용할 때 E4B의 이름을 파일 크기 4GB로 해석하면 안 됩니다. 카드의 Q8 가중치는 약 8.01GB이며 별도 시각 파일은 추가 용량입니다. Q8과 BF16의 보정 온도가 서로 다르게 제시되어 있으므로 배포 정밀도와 설정을 함께 기록해야 합니다.

근거 [S15]

업무별 어댑터가 적합한 경우

자주 반복되는 업무에 충분한 정답 데이터가 있다면 작은 모델에 그 업무를 더 가르치는 방법을 고려할 수 있습니다. 다만 먼저 데이터에 쉽게 외울 수 있는 표현이나 중복이 없는지 살펴야 합니다. 회사 이름이나 문서 양식만 외워 좋은 점수를 낸다면 새 문서에서 쉽게 실패합니다.

12 Nimble과 Jeeves의 다른 선택

Bespoke Nimble 9B v2

구조 Qwen3.5 9B + LoRA → 스키마에 맞는 후보 답 토큰 점수

스키마는 어떤 질문과 답 형식을 받을지 미리 정한 약속입니다. Nimble은 이 약속에 맞춘 텍스트 판단을 하며 이유를 생성하지 않습니다. 약 173MB의 어댑터 파일만으로 실행되는 것이 아니라 별도의 Qwen3.5-9B 본체가 필요합니다. 현재 배포는 최대 255개 후보를 지원하되 학습에서 본 후보 수와는 다릅니다.

사용할 때 공식 로더의 현재 기본 보정 온도는 약 2.179입니다. 이는 v2에 새로 맞춘 값이 아니라 이전 설정에서 가져온 추론 기본값이라고 카드가 명시합니다. 가중치만 로드하면 보정이 빠질 수 있으며, 후보 수 지원 확대가 그 범위의 정확도 검증을 뜻하지는 않습니다.

근거 [S16]

Jeeves PostHog

구조 Qwen3.5 9B + LoRA → 포인터 헤드 · 선택적으로 생각을 먼저 생성

256차원의 질문·선택지 투영을 쓰는 포인터 헤드와 보정값을 함께 배포합니다. 지도 학습과 CISPO 강화 학습을 설명하며, 생각하는 경로의 속도를 높이기 위한 추측 디코딩 보조 모델도 제공합니다. 생각을 생략하는 경로와 먼저 생각하는 경로가 구분됩니다.

사용할 때 모든 Decision 모델이 반드시 출력 토큰을 전혀 만들지 않는 것은 아닙니다. 어떤 경로를 사용했는지 기록해야 정확도와 속도를 비교할 수 있습니다. 전용 코드, 헤드, 프롬프트를 함께 사용해야 하며 일반 모델 파일만 불러오는 것과 같지 않습니다.

근거 [S17]

비교 설정에 반드시 남길 것

추론을 켰는지, 어떤 헤드를 썼는지, 보정 온도가 적용됐는지까지 기록해야 합니다. 같은 모델명이라도 이 설정들이 다르면 전혀 다른 실험이 됩니다. 지원 가능한 선택지 수와 실제로 평가된 선택지 수 역시 구분해서 읽으세요.

13 Clef와 Von의 선택지 처리

Clef Flash Cloudflare

구조 Qwen3.5 9B와 시각 인코더 → 작은 Transformer형 joint schema head

본체가 읽은 최종 표현에서 각 질문에 필요한 정보를 모으고, 여러 질문의 모든 선택지를 함께 점수화하는 전용 헤드를 사용합니다. 질문마다 softmax를 적용합니다. 원 배포는 이미지와 영상 처리 구성요소도 포함하며 더 큰 Clef 변형과 구분됩니다.

사용할 때 단순히 Qwen3.5-9B의 A/B 확률을 읽는 방식과 다릅니다. joint_head 파일과 전용 입력 구성 코드가 필요합니다. 멀티모달 지원과 여러 질문 공동 처리는 원 실행기의 기능이며 현재 텍스트 전용 앱에 저절로 추가되지 않습니다.

근거 [S18]

Von

구조 ModernBERT-large → 선택지별 MASK 표시 → 작은 점수 헤드

약 395M 본체 기반이며 현재 엔진 1.3은 1.2의 가중치를 사용한다고 명시합니다. 1.2부터 각 후보가 공통 상황과 자기 후보만 보도록 attention을 제한하고 위치를 재설정합니다. 선택지 순서를 바꿨을 때 점수가 바뀌는 문제를 줄이려는 구조입니다.

사용할 때 기본 판단 확률에는 보정 외의 정책도 개입할 수 있습니다. 카드의 Noul band rule처럼 중간 구간 점수를 바깥으로 옮기는 동작은 원래 분포와 다릅니다. 순수 확률을 비교하려면 raw 경로와 후처리를 확인해야 합니다. 순서에 안정적인 설계가 정답률까지 보장하는 것은 아닙니다.

근거 [S19]

선택지끼리 서로 영향을 줄 수 있습니다

예를 들어 회계, 기술, 기타를 넣었을 때와 회계, 기술, 환불, 기타를 넣었을 때 회계 점수는 달라질 수 있습니다. 후보 설명이 본체의 문맥을 바꾸기도 하고, 같은 점수라도 softmax의 분모가 바뀌기 때문입니다. 따라서 선택지를 추가한 뒤에는 기준값도 다시 검증해야 합니다.

14 기존 분류기를 함께 비교하는 이유

GLiClass 계열

구조 문서와 라벨 설명을 표현으로 변환 → 라벨별 분류 점수

라벨을 사람이 읽는 설명으로 주어 새로운 분류 이름에도 대응하는 계열입니다. 하나의 인코더에 함께 넣는 uni encoder, 문서와 라벨을 따로 읽는 bi encoder, 이를 결합하는 변형 등 구조가 여럿입니다. 특정 체크포인트의 본체와 설정을 확인해야 합니다.

사용할 때 다중 선택 모드에서는 여러 라벨이 동시에 맞을 수 있습니다. 이때 점수 합이 1이어야 하는 단일 선택 softmax와 다릅니다. “개인정보도 있고 계약정보도 있음” 같은 겹치는 분류에 유용할 수 있지만, 현재 앱의 기밀/비기밀 분포와 그대로 합쳐 비교하면 안 됩니다.

근거 [S20]

NLI 기반 제로샷 분류

구조 문서와 라벨을 설명하는 가설 문장 → 인코더 → 가설 성립 점수

NLI는 한 문장이 다른 문장을 뒷받침하는지 판단하는 과제입니다. “이 문서는 계약 조건을 담고 있다”를 가설로 만들고 원문이 이를 뒷받침하는지 계산해 분류에 활용합니다. MoritzLaurer의 ModernBERT 및 DeBERTa 계열은 이런 비교의 후보가 됩니다.

사용할 때 후보마다 문서와 가설의 쌍을 만들어 평가하는 구현이 일반적입니다. 한 번의 API 요청으로 묶었다고 문서를 딱 한 번만 읽는 전용 헤드와 같은 계산이 되는 것은 아닙니다. 가설의 표현과 언어, 다중 라벨 여부에 따라 점수 해석도 달라집니다.

근거 [S21, S25]

새 이름이 붙은 모델만 볼 필요는 없습니다

문서가 짧고 분류가 명확하면 오래된 구조의 분류기도 좋은 비교 기준이 됩니다. 반대로 숫자 조건을 여러 단계로 연결해야 하는 규칙이라면 빠른 분류기보다 생각하는 모델이나 코드 규칙이 적합할 수 있습니다. 업무의 난도와 실패 유형을 먼저 정의해야 구조를 선택할 수 있습니다.

15 CLM과 모델 바깥의 실행 도구

CLM v0.1 8B

구조 고정된 Qwen3 8B 표현 → 상태 헤드와 행동 헤드 → 두 표현의 일치 점수

본체는 고정하고 상태와 행동을 위한 작은 투영 헤드를 대조 학습합니다. 맞는 상태·행동 조합을 가깝게, 맞지 않는 조합을 멀게 만드는 방식입니다. 상태와 후보 행동을 따로 읽으므로 반복 사용하는 행동의 표현을 재사용할 수 있습니다.

사용할 때 많은 행동 후보를 순위화할 때 고려할 구조입니다. 반환 확률은 주어진 후보 집합에 대한 상대 값이며, 공개된 에이전트 성능 중 일부는 업무별로 다시 학습한 헤드의 결과입니다. 모델 카드의 양방향 InfoNCE는 손실 방향을 뜻하며, Qwen 본체가 양방향 attention 인코더로 바뀌었다는 뜻은 아닙니다.

근거 [S22]

래퍼와 모델 가중치를 구분하기

이름 또는 종류	담당하는 역할
Privatemode Decisions	일반 LLM의 선택지 토큰 확률을 읽는 라이브러리
SemIf OpenJev 계열	질문 구성과 상태 계산 재사용, 선택지 평가를 구현
Ollama	모델 파일을 관리하고 채팅 등 추론 API를 제공
Ollaya	여러 Decision 모델의 고유 템플릿과 실행을 제공
판단 실험실 앱	문서 입력, 로컬 연결, 점수 해석, 비교 화면을 제공

래퍼는 모델을 부르는 방법을 바꾸는 코드입니다. 같은 인터페이스를 제공하더라도 안에서 기존 답 토큰, 포인터 헤드, 인코더를 사용할 수 있습니다. 따라서 System One 호환 API라고 모두 Jev와 같은 구조나 보정 특성을 갖는 것은 아닙니다. [S3, S24]

이번 리포트의 17개 계열은 공개 설명이 있는 주요 구조를 폭넓게 비교한 목록입니다. 각 크기·양자화·업무용 어댑터를 모두 별개의 새 구조로 세지 않았습니다. 새 변형을 선택할 때는 본체, 헤드, 템플릿, 보정, 지원 언어를 다시 확인하는 방식으로 이 목록을 확장할 수 있습니다.

16 점수와 보정과 기준값을 구분하기

점수

0.8이라는 숫자가 나왔다고 의미가 항상 같지는 않습니다. A/B 확률을 다시 나눈 값일 수도 있고, 전용 분류 헤드의 확률일 수도 있으며, AI가 설명문과 함께 스스로 써낸 위험도일 수도 있습니다. 마지막 방식은 토큰 확률과 직접 비교할 수 없습니다.

보정 Calibration

많은 문서에서 0.8 정도를 준 문서를 모았을 때 실제로 약 80퍼센트가 해당 조건을 만족한다면 그 구간의 눈금이 잘 맞는 것입니다. 한 문서의 안전을 보장하는 말이 아니라 예측들을 모은 집단의 성질입니다. 새 언어, 새 업무, 새 질문에서는 눈금이 달라질 수 있습니다. [S23]

보정 온도 T가 1이면 점수 차이를 그대로 둡니다. T가 1보다 크면 분포가 부드러워지고, 작으면 한쪽에 더 몰립니다. 양의 공통 온도 보정은 가장 큰 후보의 순서를 바꾸지 않지만, 고정된 운영 기준을 넘는지는 바꿀 수 있습니다.

기준값 Threshold

기밀 점수	현재 시작용 규칙	실무에서의 의미
0.25 미만	반출 가능 후보	기밀 가능성이 낮다고 본 후보
0.25 이상 0.60 미만	사람 검토	원문과 상황을 사람이 확인
0.60 이상	반출 금지	기밀 위험이 높다고 분류

기준값은 업무의 실수 비용을 반영합니다. 기밀을 놓치는 실수가 크면 검토를 늘릴 수 있습니다. 하지만 모든 문서를 잡으면 검토 비용이 커집니다. 이 균형은 모델 점수만으로 자동 결정되지 않습니다. 앱은 시작값을 제공할 뿐 회사 규정에 맞춘 기준을 이미 학습한 것이 아닙니다.

확신이 높아도 틀릴 수 있습니다

분포가 한쪽에 몰린 정도를 confidence라는 별도 값으로 표현하는 도구도 있습니다. 이 값은 정답 여부 자체가 아니며, 단순히 선택된 후보의 확률과도 다를 수 있습니다. 대시보드의 점수 막대는 모델의 출력을 설명하는 도구로 읽어야 합니다. 실제 정답을 붙인 문서가 있어야 맞았는지 평가할 수 있습니다.

17 공정한 비교를 위한 지표와 절차

정확도 하나만 보면 중요한 실수를 숨길 수 있습니다. 문서 1,000건 중 기밀이 10건뿐일 때 모두 공개라고 답해도 정확도는 99퍼센트입니다. 하지만 기밀은 전부 놓쳤으므로 실제 업무에는 부적합합니다. 이 예는 개념 설명용 가상 수치입니다.

볼 지표	현재 앱의 계산	읽는 방법
기밀을 놓침	기밀 정답인데 가능 후보가 된 수 / 완료 기밀 수	낮을수록 좋음. 검토와 차단은 일단 붙잡은 것으로 셈
공개를 붙잡음	공개 정답인데 검토·차단된 수 / 완료 공개 수	불필요한 검토와 제동이 얼마나 생기는지
사람 검토	검토 분류 수 / 성공적으로 분석한 수	사람에게 넘어올 업무량
분석 오류와 미실행	별도 건수	누락된 결과를 성공처럼 세지 않음
처리 시간 중앙값	시간이 제공된 완료 결과들의 가운데 값	처음 준비 시간 및 전체 요청 시간과 별도

이 지표는 정확히 맞힌 기밀 여부와 완전히 같지는 않습니다. 예를 들어 기밀을 사람 검토로 보낸 것은 “기밀을 놓침”에는 들어가지 않지만 최종 정답을 확정한 것도 아닙니다. 오류율과 검토량을 함께 보아야 모든 것을 검토로 보내는 모델을 좋은 모델로 착각하지 않습니다.

평가를 세 단계로 나누세요

먼저 실제 업무 문서에 사람이 정답을 붙입니다. 서로 거의 같은 양식이나 같은 사건의 문서는 한 묶음으로 다룹니다. 다음으로 일부 문서에서 질문과 보정, 기준값을 조정합니다. 마지막으로 아직 사용하지 않은 문서에서 성능을 측정합니다. 가능하면 보정용과 기준 선택용 데이터도 분리하거나 교차 검증을 설계합니다.

일반 문서뿐 아니라 공개 교육 자료, 인용, 긴 문서, 띄어쓰기 오류, 숫자 조건, “이전 지시를 무시하라” 같은 원문 내 지시도 평가에 넣습니다. 선택지 순서를 바꾸거나 같은 요청을 반복해 흔들림도 확인하세요. 문서 분류를 유도하는 문장은 모델에 대한 정상 지시가 아니라 분류 대상 원문으로 취급해야 합니다.

18 이 PC에서 측정한 체험 결과

아래는 앱의 가상 예제 4건을 네 모델에서 실제 실행한 저장 결과입니다. 2026년 10월 4일 17시 39분 실행 기록을 사용했습니다. 기밀 2건과 공개 2건뿐이므로 제품 전체의 성능 순위나 실무 정확도로 일반화하면 안 됩니다.

문서	정답	Qwen	JevK5	Laya	decider
미공개 계약 조건	기밀	0.998	0.976	0.974	0.971
공개 행사 안내	공개	0.012	0.003	0.982	0.088
헷갈리는 보안 교육	공개	0.009	0.001	0.963	0.106
표시 없는 내부 계획	기밀	0.988	0.958	0.849	0.725

숫자는 기밀 판단 점수입니다. 모든 모델의 검토 시작 0.25, 차단 시작 0.60. 모델마다 점수의 눈금이 같다는 뜻은 아닙니다.

모델	기밀 누락	공개 제동	추론 중앙값
Qwen3.5 · 4B	0 / 2	0 / 2	115.5 ms
JevK5 · 4B	0 / 2	0 / 2	89.2 ms
Laya · 다국어	0 / 2	2 / 2	27.9 ms
decider · 0.8B	0 / 2	0 / 2	179.1 ms

이 네 문서에서는 Qwen, JevK5, decider가 기밀을 잡고 공개 문서를 후보로 두었습니다. Laya는 네 문서를 모두 차단했고 공개 문서 두 건에서도 높은 점수를 냈습니다. 현재 질문과 실행 조건에서 Laya의 눈금과 문맥 구분을 더 확인해야 한다는 신호입니다.

시간은 모델 실행기가 보고한 추론 시간이며 네 건의 중앙값입니다. 네트워크 포함 서비스 지연이나 앱 첫 실행 시간과 다릅니다. Qwen은 Q4_K_M, JevK5는 Q8_0, 나머지는 ONNX 실행을 사용하므로 공통 하드웨어 위에서도 모든 계산 조건이 같지는 않습니다.

원본 기록: verification/example-comparison.json. 배포 식별값, 질문, 시간, 모델 정보가 포함됩니다. 점수는 소수 셋째 자리, 시간은 밀리초 한 자리로 반올림했습니다.

19 바뀐 화면으로 비교하는 방법

모델의 역할부터 고르기

모델 비교 화면은 Qwen을 일반 AI 활용, JevK5를 판단 전용 학습, Laya를 짧은 문서 분류, decider를 소형 판단 AI로 설명합니다. 카드의 구조와 사용 시 주의점을 펼치면 본체와 점수 읽는 방법, 언어·입력 한도의 차이를 볼 수 있습니다. 이는 성능 순위가 아니라 비교 목적입니다.

모든 모델에 같은 문서와 질문 주기

처음에는 예제 4건을 선택해 실행합니다. 이후 내 문서 한 건을 붙여넣거나 정답을 붙인 JSON 문서 묶음을 불러옵니다. 예제 질문을 바꾸면 원래 붙어 있던 예제 정답은 적용하지 않습니다. 사용자가 올린 데이터셋은 질문과 정답이 맞는지 직접 확인해야 합니다.

한 문서씩 나란히 보기

결과 상단에서 문서를 선택하면 모든 모델의 점수 막대, 최종 분류, 사람이 붙인 정답과의 관계, 처리 시간 막대가 함께 보입니다. 작은 화면에서는 카드가 세로로 놓여 가로 스크롤 없이 비교할 수 있습니다. 막대의 검토·차단 표시를 보면 같은 점수가 왜 다른 행동으로 바뀌는지도 읽을 수 있습니다.

보안 교육 예제를 보면 기밀이라는 단어가 있는 공개 문서에서 모델이 과하게 막는지 비교하기 좋습니다. 정답을 지정하지 않은 문서는 맞고 틀림을 평가하지 않고 점수와 시간만 보여줍니다. 시간 막대가 짧을수록 그 문서의 모델 처리 시간이 짧습니다.

묶음 전체의 실수와 누락 확인하기

아래 요약에서는 모든 모델의 결과가 모인 문서 중 분류가 갈린 건수를 보여줍니다. 각 모델의 기밀 누락, 공개 문서 제동, 검토량, 시간 중앙값과 오류를 함께 확인합니다. 전체 상세표는 펼쳐 볼 수 있고 원문과 질문도 확인할 수 있습니다.

기준을 바꾸고 결과를 보관하기

모델 카드의 기준 설정을 펼쳐 값을 바꾸면 이미 계산된 점수로 다시 분류합니다. 재추론은 하지 않습니다. “결과 JSON 받기”는 원문을 제외한 질문, 문서 이름, 정답, 모델 식별 정보와 결과를 저장합니다. 질문과 문서 이름에도 민감한 내용이 있을 수 있으므로 파일을 공유하기 전 내용을 확인하세요.

20 앱 구조와 파일의 역할

앱은 브라우저 화면, Python 앱 서버, 모델 실행 도구의 세 부분으로 나뉩니다. 브라우저가 문서와 질문을 모아 앱 서버로 보내고, 앱 서버가 로컬 모델 실행기에 요청합니다. 모델이 반환한 결과를 다시 화면에서 비교합니다.

파일 또는 폴더	역할	수정하는 상황
dist/index.html	입력창과 비교·학습 화면의 뼈대	메뉴, 설명 문구, 화면 항목 추가
dist/styles.css	배치, 색, 글자, 좁은 화면 대응	카드 배치나 시각 표현 변경
dist/app.js	한 문서 분석과 설정 관리	문서 입력, 연결, 결과 표시 변경
dist/compare.js	여러 모델의 순차 실행과 비교 화면	모델 카드, 묶음 실행, 내보내기 변경
dist/comparison-metrics.js	분류·집계·중앙값 계산	지표와 계산 규칙 변경
server.py	요청 검사와 모델별 응답 변환	연결 방식, 검증, API 추가
run.py와 start.command	로컬 실행 도구와 앱 시작	실행 경로, 프로세스 관리 변경
tests와 verification	계산 검증과 실제 실행 기록	회귀 확인, 결과 재현
report	종합 리포트와 출처 자료	문서 갱신

화면과 서버 사이의 약속

GET /api/health는 앱이 실행 중인지 확인합니다. POST /api/connect는 실행 도구의 모델 목록을 읽습니다. POST /api/analyze는 문서와 질문을 보내고 점수와 판단, 소요 시간을 돌려받습니다. 입력에는 provider, model, text, criterion, low, high 같은 필드가 포함됩니다.

여기서 provider는 AI 모델 이름이 아니라 모델을 실행하는 도구입니다. 예를 들어 Ollaya라는 실행 도구가 JevK5, Laya, decider라는 서로 다른 모델을 실행합니다. 모델을 바꾼다는 것과 서버 프로그램을 바꾼다는 것은 구분해서 이해해야 합니다.

21 서버 안에서 문서가 처리되는 순서

1 요청을 검사합니다

Handler가 로컬 주소와 요청 헤더를 확인합니다. config와 local_url은 연결 대상이 이 PC의 HTTP 서버인지, 숫자 기준이 유효한지 검사합니다. 문서는 최대 12,000자이고 문서와 질문을 합쳐 UTF-8 24KB 이내여야 합니다. 문자 수가 같아도 언어마다 바이트와 토큰 수가 달라집니다.

2 우선 규칙을 적용하거나 모델을 부릅니다

한 문서 분석의 우선 규칙이 켜져 있으면 rule_matches가 주민등록번호, 비공개 키, 접근 키 형태를 먼저 찾습니다. 형태만 검사하므로 가짜 예시도 걸릴 수 있습니다. 모델 비교는 이 규칙을 꺼서 모델 자체의 차이를 봅니다. INFERENCE_LOCK은 동시에 여러 추론이 겹치는 것을 막습니다.

3 실행 도구별 응답을 점수로 바꿉니다

Ollama와 로컬 호환 서버의 tokens 방식은 normalize_logprobs가 A/B의 로그확률을 정규화합니다. 공백이 붙은 변형을 모으고, 두 후보가 없으면 오류로 처리합니다. Ollama 모델의 원격 실행 메타데이터도 검사하여 클라우드 모델을 차단합니다.

Ollaya의 decision 방식은 /api/decide에 state와 noul 질문을 보내 예 점수를 받습니다. 문서가 잘렸거나 완료 이유가 예상과 다르면 유효한 결과로 쓰지 않습니다. 내장 보정을 또 한 번 적용하지 않습니다. 설명 생성 방식은 원문에 실제 존재하는 근거만 표시합니다.

4 행동과 시간을 돌려줍니다

classify는 우선 규칙과 불확실성을 먼저 반영하고, 그 다음 low와 high로 행동을 나눕니다. timings는 실행기가 제공한 준비·추론 시간을 초로 바꿉니다. 제공되지 않은 시간은 0으로 꾸미지 않습니다. 브라우저는 받은 결과를 메모리에 두고 화면에 보여줍니다.

오류가 생겼을 때

연결 실패, 잘못된 점수, 입력 잘림을 가능 후보로 바꾸지 않습니다. 단일 문서 화면은 검토 대상으로 남기고, 비교 화면은 분석 오류로 표시해 점수 비교에서 제외합니다. 오류와 미실행 수는 분모 밖으로 사라지지 않도록 별도로 표시합니다.

22 비교 코드와 확장 방법

compare.js의 역할

profiles에 네 모델의 이름, 실행 도구, 방법, 기준값을 정의합니다. checkModels가 실제 준비 상태를 확인합니다. runComparison은 문서와 질문을 복사한 실행 기록을 만들고 모델별로 문서를 차례로 처리합니다. 실행 중 입력을 잠가 서로 다른 질문의 결과가 한 묶음에 섞이지 않게 합니다.

중단 버튼은 진행 중인 모델 호출이 끝난 뒤 다음 요청을 멈춥니다. 이미 완료된 결과는 남고 나머지는 미실행으로 표시됩니다. 이것은 현재 추론을 강제 취소하는 기능과 다릅니다. 렌더링은 점수 막대와 문서별 상세표, 묶음 요약을 같은 기록에서 만듭니다.

comparison metrics의 역할

outcome은 점수와 기준값으로 행동을 계산합니다. modelSummary는 정답이 있는 완료 결과로 누락과 제동을 셉니다. agreement는 선택한 모든 모델에서 유효한 결과가 있는 문서만 비교합니다. median은 시간이 실제로 제공된 값만 사용합니다. DOM에 의존하지 않는 계산이므로 별도 테스트에서 검증할 수 있습니다.

다른 모델을 더 넣으려면

먼저 모델을 지원하는 로컬 실행기가 있는지 확인합니다. 같은 Ollaya 응답 계약으로 실행되는 모델이면 profiles에 식별자와 설명을 추가하고 모델 목록에서 실제 태그를 확인하는 경로가 출발점입니다. 준비된 모델 수 같은 고정 표시도 동적으로 바꾸어야 합니다. 다른 API라면 서버에 별도 변환기를 추가하고 점수·잘림·오류 검사를 맞춰야 합니다.

모델 파일만 설치하고 기존 점수 의미를 그대로 가정해서는 안 됩니다. 포인터 헤드 파일, 스키마, 온도와 후처리, 지원 언어, 입력 한도를 확인하고 작은 정답 자료로 먼저 실행해 봅니다. 현재 앱의 비교 목록은 네 종으로 고정되어 있습니다.

다른 업무로 넓히려면

예를 들어 문의 부서를 고르는 앱은 기밀이라는 불리언 대신 회계·기술·고객상담의 분포가 필요합니다. 입력 스키마, 서버의 질문, 응답 변환, 화면의 결과명, 정답 집계와 테스트를 함께 바꿔야 합니다. 지금 화면에서 질문만 바꾸면 기밀 기준의 집계와 이름이 남으므로 범용 분류 앱이 되지는 않습니다.

검증 범위: 서버의 주소·확률·잘림 등 핵심 검사와 비교 집계 테스트, 실제 네 모델 실행을 별도로 확인합니다. 모델의 미래 정답률을 보장하는 검증은 아닙니다.

23 자주 헷갈리는 용어

용어	쉬운 설명
매개변수 Parameter	학습으로 조정된 수치. B는 10억, M은 100만 단위. 크기만으로 성능을 결정하지 않음
가중치 Weights	학습된 매개변수를 저장한 모델 파일
토크나이저 Tokenizer	문장을 모델이 읽는 조각과 번호로 바꾸는 도구
Logit	확률로 바꾸기 전의 비교 점수. 음수나 1보다 큰 값도 가능
Softmax	선택지 점수들을 합이 1인 분포로 바꾸는 계산
Log probability	확률의 로그 값. 작은 확률을 안정적으로 다루는 데 유용
Prefill	출력을 쓰기 전에 입력 문맥을 읽는 계산
Decode	읽은 문맥 뒤에 출력 토큰을 이어 쓰는 계산
헤드 Head	본체의 내부 표현을 답이나 분류 점수로 바꾸는 마지막 장치
양자화 Quantization	가중치를 더 적은 비트로 표현해 용량을 줄이는 방법. 점수에도 영향을 줄 수 있음
GGUF와 ONNX	모델 저장·실행에 쓰는 형식. 그 자체가 모델의 판단 구조나 정확도 이름은 아님
Calibration	예측한 확률과 실제 발생 빈도가 맞도록 눈금을 확인·조정하는 것
Threshold	점수를 검토·차단 같은 행동으로 바꾸는 경계
Zero shot	해당 분류 과제의 예시를 추가로 학습하지 않고 설명만으로 수행하는 설정
Holdout	기준 조정에 쓰지 않고 마지막 채점까지 남겨 둔 데이터
Digest	받은 모델 파일이나 배포를 식별하는 값. 이름이 같은 다른 버전을 구분할 때 사용

실무 선택은 “가장 큰 모델”이나 “가장 빠른 모델”을 고르는 문제가 아닙니다. 허용할 기밀 누락, 사람이 검토할 수 있는 양, 문서 길이와 언어, 필요한 응답 시간을 정한 뒤 같은 자료로 후보를 비교하는 과정입니다.

24 참고 자료와 근거 1

모델 구조는 개발자 모델 카드와 공식 문서, 실험 해석은 영상과 원문 및 공개 구현을 기준으로 했습니다. 웹 문서는 갱신될 수 있습니다. 아래 번호는 본문의 근거 표시와 연결됩니다. 확인일은 2026년 10월 4일입니다.

S1 단테랩스 영상과 실험 한계

S2 추가 글 원문 Privatemode

S2a GeekNews 소개와 토론

S3 Privatemode Decisions 구현

S4 Privatemode 공개 평가 구현

S5 TypeSafe System One 개념

S6 TypeSafe 학습과 보정 개념

S7 Qwen3.5 4B 모델 카드

S8 GLM 5.3 Flash 모델 카드

S9 JevK5 모델 카드

S10 Laya 계열 모델 카드

S11 decider 0.8B 모델 카드

S12 Kev 4B 모델 카드

영상의 주요 구간: 15분 43초 전후 결과, 21분 55초 전후 평가 한계. 이 앱의 체험 결과는 영상과 별개입니다.

25 참고 자료와 근거 2

S13 Eos 공개 구조 문서

S14 Jeff Firelex 프로젝트

S15 Winnow E4B 모델 카드

S16 Bespoke Nimble 9B v2 모델 카드

S17 PostHog Jeeves 모델 카드

S18 Cloudflare Clef Flash 모델 카드

S19 Von 모델 카드

S20 GLiClass 공식 구현

S21 ModernBERT 제로샷 분류 모델 카드

S22 CLM v0.1 8B 모델 카드

S23 scikit learn 확률 보정 설명

S24 SemIf OpenJev 구현

S25 Hugging Face 제로샷 분류 구현

로컬 결과와 코드 확인

실제 측정 자료는 decision-lab/verification/example-comparison.json에, 화면과 서버 코드는 같은 폴더의 dist 및 server.py에 있습니다. 모델 다운로드 폴더와 보고서 출처 자료는 서로 다른 목적의 파일입니다. 앱 화면의 종합 리포트 메뉴에서도 이 문서를 받을 수 있습니다.